



Universidade Federal do Ceará – Campus de Quixadá

Disciplina: QXD0043 - Sistemas Distribuídos

Cursos: SI, RC, ES, CC, EC e DD

Professor: Antonio Rafael Braga

### Trabalho 1 – Comunicação entre processos (Capítulo 4)

**Cada estudante ou dupla deve definir um serviço remoto. Crie pelo menos 2 classes do tipo POJO de acordo com o serviço escolhido que representem as informações que existem do serviço escolhido e 2 classes de serviço que implementam operações.**

#### ***Sockets e Streams***

**1. Escolha uma das classes POJO do serviço remoto selecionado e transforme essa classe uma subclasse de `OutputStream`, *por exemplo*, *PojoEscolhidoOutputStream* que envia os dados de um conjunto (array) de qualquer outro POJO criado anteriormente seguindo a seguinte regra:**

- a) O construtor da subclasse deve receber como parâmetros (i) um `OutputStream` de destino, para onde as informações do conjunto de objetos devem ser enviadas (ii) um array de objetos que representam os dados a serem transmitidos; (iii) o número de objetos que terão dados enviados pelo *stream*;
- b) Teste sua implementação utilizando como destino a saída padrão (`System.out`);
- c) Teste sua implementação utilizando como destino um arquivo (`FileOutputStream`);
- d) Teste sua implementação utilizando como destino um servidor remoto (TCP).

**2. Crie uma subclasse para `InputStream` chamada *PojoEscolhidoInputStream* que lê os dados gerados pelo stream do exercício anterior.**

- a) O construtor da subclasse deve receber como parâmetro um `InputStream` de origem, de onde as sequências de bytes serão lidas.
- b) Teste sua implementação utilizando como origem a entrada padrão (`System.in`)
- c) Teste sua implementação utilizando como origem um arquivo (`FileInputStream`).
- d) Teste sua implementação utilizando como origem um cliente remoto (TCP).

#### **Serialização (Representação externa de dados)**

**3. Implemente no serviço remoto escolhido uma Serialização de um objeto e transmita esse objeto por meio de uma comunicação cliente-servidor. A comunicação entre cliente e servidor deve ser implementada via *sockets* (TCP ou UDP) que trocam fluxos de bytes.**

- a) As estruturas de dados devem ser ‘serializadas’ para serem enviadas em mensagens, ou seja, deve ser feito o empacotamento e desempacotamento das mensagens no lado cliente e no lado servidor, ou seja:

- O cliente deve empacotar a mensagem de **request** antes de enviar para o servidor;
- O cliente deve desempacotar a mensagem de **reply** enviada pelo servidor;
- O servidor deve desempacotar a mensagem de **requisição** do cliente;
- O servidor deve empacotar a mensagem de **reply** e enviar para o cliente;

## Multicast

### 4. Implemente no serviço remoto escolhido uma comunicação *multicast*, onde:

- O endereço IP do servidor é um endereço IP classe D (ex: 230.0.0.1) e possui um porta fixa. Clientes devem ser capazes de: entrar no grupo (`joinGroup`) e sair do grupo (`leaveGroup`);
- O servidor deve enviar mensagens *multicast* para todos os clientes conectados. Exemplos de mensagens: NOTIFICAÇÃO, ALERTA e ATUALIZAÇÃO; Formato sugerido (JSON ou binário):  

```
{
  "tipo": "NOTIFICACAO",
  "mensagem": "Novo dado disponível",
  "timestamp": 123456789
}
```
- Implementar múltiplos clientes que recebem mensagens simultaneamente e exibem mensagens em tempo real;
- Os clientes devem: (i) ter uma *thread* para escutar o multicast, (ii) outra *thread* para a interação com usuário. O servidor deve: suportar envio concorrente de mensagens (*multithreading*).
- Use *sockets* TCP (*unicast*) para a autenticação de clientes e *sockets* UDP (*multicast*) para o envio de das mensagens.

### Os seguinte critérios de avaliação serão aplicados:

- Correção funcional (20%)
- Organização do código (20%)
- Uso adequado de conceitos (20%)
- Testes e demonstração (20%)
- Documentação do repositório (20%)

### Questão extra

6. Implemente uma aplicação distribuída que suporte um sistema de votações. O envio de votos têm um prazo máximo, finalizado esse tempo o servidor não aceita mais votações e calcula o total de votos, respectivas percentagens e candidato ganhador. O eleitor começa a votação através de um login. Em resposta, o servidor envia-lhe uma lista de candidatos que estão em votação, o que permitirá ao eleitor votar em um determinado candidato. Além dos eleitores, existem também os administradores do sistema que têm a capacidade de introduzir e remover candidatos para votação e que poderão enviar notas informativas para os eleitores.

a) O login, envio da lista de candidatos e de votos deverá ser implementada usando comunicação *unicast* com API de *sockets* TCP, enquanto que toda a comunicação *multicast* será feita utilizando *sockets* UDP. O multicast será utilizado exclusivamente para as notas informativas enviada pelos administradores do sistema. O servidor deverá ser *multithreading*. Para a representação externa de dados nas chamadas remotas (métodos, argumentos e resultados), sugere-se que seja implementada através de *protocol buffers*. Contudo, versões em **XML** e **JSON** também são aceitas.

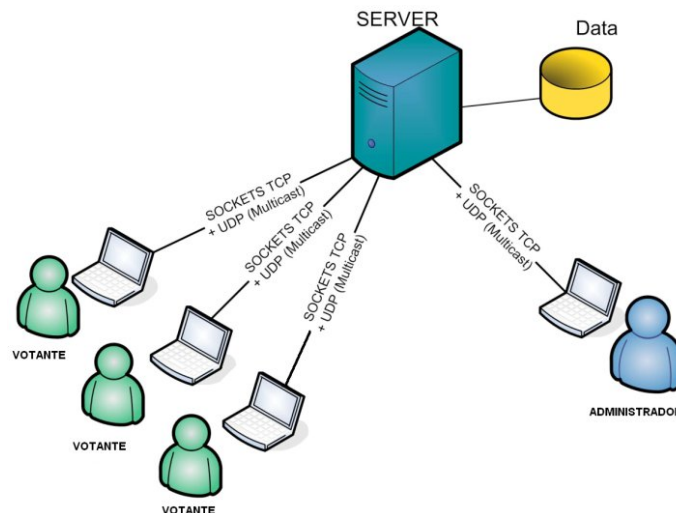


Figura 1: Arquitetura da aplicação, utilizando Sockets

Bom trabalho!